

Landmålerberegner

Læremateriale om vibe coding, React, kort og danske geodata-API'er

Dette dokument opsummerer det, du har lært undervejs i byggeriet af din browserbaserede landmåler-app: fra Git og VS Code, over React og Leaflet-kort, til koordinatsystemer og integration med Dataforsyningens API'er. Brug det som opslagsværk, når du bygger videre.

Indhold

1. Vibe coding — grundtanken
2. Værktøjer: VS Code, Node.js, npm, terminal
3. Git — versionsstyring
4. Projektopsætning: Vite + React
5. React — grundbegreber
6. Kort med Leaflet
7. Koordinatsystemer i Danmark
8. Arbejde med eksterne API'er sikkert
9. GraphQL — grundbegreber
10. Projektstruktur: utils vs. components
11. Fejlfinding — typiske faldgruber
12. Kommando-opslag (hurtig reference)
13. Ordliste

1. Vibe coding — grundtanken

"Vibe coding" er en arbejdsform, hvor du bygger software i en løbende dialog med en AI-assistent: du beskriver, hvad du gerne vil have, assistenten skriver koden, du tester den i praksis, og I justerer sammen trin for trin. Du behøver ikke kunne skrive al koden selv — men det hjælper at forstå strukturen, så du kan give præcis feedback og fejlsøge sammen med assistenten.

Gode vaner fra dette projekt:

- Commit til Git ofte — efter hver fungerende ændring, ikke kun ved store milepæle.
- Test én ting ad gangen, før I går videre til det næste.
- Når noget fejler: kopiér den fulde fejlbesked ind i samtalen — jo mere præcist, jo hurtigere fejlsøgning.
- Vær ikke bange for at bede om at få en hel fil igen, hvis mange små rettelser gør det uoverskueligt.
- Bed om at få omstruktureret koden i flere filer, når én fil vokser sig for stor og uoverskuelig.

2. Værktøjer: VS Code, Node.js, npm, terminal

VS Code er editoren, hvor du redigerer kodefiler. **Node.js** er programmet, der kører JavaScript uden for browseren — det bruges til at installere pakker og køre udviklingsserveren. **npm** (Node Package Manager) følger med Node.js og bruges til at installere biblioteker (pakker) som React, Leaflet og proj4.

Tjek at værktøjerne er installeret:

```
node -v
npm -v
git --version
```

Terminalen i VS Code åbnes med **Terminal** → **New Terminal**, eller genvejen Ctrl+`. Alle kommandoer i dette dokument køres i terminalen, som udgangspunkt inde i din **surveyor-app**-mappe (der hvor package.json ligger).

3. Git — versionsstyring

Git tager "øjebliksbilleder" (commits) af din kode, så du altid kan gå tilbage til en tidligere fungerende version. GitHub er en cloud-tjeneste, hvor du kan gemme dit Git-repository sikkert og tilgå det fra andre maskiner.

De vigtigste kommandoer:

Kommando	Hvad den gør
git init	Starter et nyt Git-repository i den aktuelle mappe

<code>git status</code>	Viser hvilke filer der er ændret/ikke committed
<code>git add .</code>	Forbereder ("stager") alle ændrede filer til næste commit
<code>git commit -m "besked"</code>	Gemmer et øjebliksbillede med en beskrivende besked
<code>git log --oneline</code>	Viser historikken over commits, kort format
<code>git remote add origin <url></code>	Forbinder din lokale mappe til et GitHub-repository
<code>git push</code>	Sender dine commits op til GitHub
<code>git ls-files</code>	Lister alle filer Git faktisk sporer (godt til at tjekke <code>.env</code> er IKKE med)

■■ **Vigtigt om hemmeligheder:** Filer med API-nøgler (som `.env`) må aldrig committes til Git. De skal stå i `.gitignore`. Tjek altid med `git ls-files | findstr .env` (Windows) at filen ikke bliver sporet, før du pusher til GitHub.

4. Projektop sætning: Vite + React

Vite er et moderne værktøj til at starte og køre webprojekter — det giver en hurtig udviklingsserver med automatisk genindlæsning, når du gemmer ændringer. **React** er biblioteket, vi bygger selve brugergrænsefladen i.

```
npm create vite@latest surveyor-app -- --template react
cd surveyor-app
npm install
npm run dev
```

`npm run dev` starter en lokal server, typisk på `http://localhost:5173/`. Denne skal genstartes (Ctrl+C, derefter `npm run dev` igen), hver gang du ændrer i `vite.config.js` eller i en `.env`-fil — almindelige kodeændringer i `src/`-mappen genindlæses automatisk.

5. React — grundbegreber

En React-**komponent** er en JavaScript-funktion, der returnerer, hvad der skal vises på skærmen (skrevet i en HTML-lignende syntaks kaldet JSX). Store apps bygges af mange små komponenter, der sættes sammen — ligesom vi delte Landmålerberegneren op i `MapView`, `PointsTable`, `AddressSearch` og flere andre.

useState — appens hukommelse

`useState` er en "hook", der lader en komponent huske data mellem genindlæsninger, og automatisk opdatere skærmen, når dataen ændrer sig.

```
const [points, setPoints] = useState([])
setPoints([...points, nytPunkt]) // tilføjer et punkt
```

Props — data sendt ind i en komponent

Props er, hvordan en "forælder"-komponent (fx App) sender data og funktioner videre til en "barn"-komponent (fx PointsTable), så den ved, hvad den skal vise, og hvad der skal ske ved klik.

```
<PointsTable points={points} onRemove={removePoint} />
```

useEffect — reagere på ændringer

useEffect kører kode, når noget bestemt ændrer sig — fx brugte vi det til at flyve kortet hen til et nyt punkt, hver gang flyToTarget blev opdateret.

6. Kort med Leaflet

Leaflet er et populært, gratis JavaScript-bibliotek til interaktive kort. react-leaflet er den "oversættelse" af Leaflet til React-komponenter, vi har brugt gennem hele projektet.

De vigtigste byggeklodser:

Komponent	Bruges til
MapContainer	Selve kortfladen — indeholder alt andet
TileLayer	Baggrundskortets "fliser" (Gader, Satellit, Ortofoto...)
LayersControl	Lag-vælgeren i hjørnet, til at skifte baggrundskort
Marker	Et enkelt punkt/nål på kortet
Polygon / Polyline	Flader (arealer) og linjer
CircleMarker	Et lille, farvet punkt (brugt til skelpunkter)
Tooltip	Tekstboks ved museover (evt. "permanent" for altid synlig)
Popup	Tekstboks der åbnes ved klik
useMapEvents	Hook til at lytte efter klik, højreklik m.m. på kortet
useMap	Hook til at få fat i selve kort-objektet (fx til at zoome/panorere)

7. Koordinatsystemer i Danmark

Et koordinatsystem bestemmer, hvordan et punkt på jorden beskrives med tal. Vi brugte biblioteket **proj4** til at regne mellem dem.

System	Bruges til	Enhed
WGS84 (lat/lng)	Standard for GPS og kort (Google Maps, Leaflet)	Grader
UTM Zone 32N (ETRS89)	Generel dansk/europæisk opmåling	Meter
DKTM3 (ETRS89)	Dansk ingeniør-/anlægsarbejde, zone 3	Meter
System 34 Sjælland (S34S)	Ældre, før-GPS matrikeldata (kun tilnærmet i denne app)	Meter

■■ System 34 Sjælland kræver officielt en særlig "grid-shift"-fil for præcis konvertering. Vores app bruger en forenklet tilnærmelse, som kan være flere meter unøjagtig — markeret tydeligt i appen som ikke egnet til juridisk/matrikulært brug.

8. Arbejde med eksterne API'er sikkert

Når en app skal bruge en hemmelig API-nøgle (som til Dataforsyningen), må nøglen aldrig ligge synligt i browser-koden — enhver kan åbne udviklerværktøjerne og se den. Løsningen er en **proxy**: et lille lag, der kører på din egen maskine (via Vite), modtager kaldet fra browseren, tilføjer nøglen usynligt, og sender det videre.

Mønsteret vi brugte:

- Nøglen gemmes i en .env-fil (aldrig i Git).
- vite.config.js læser nøglen og sætter en proxy op for en bestemt sti, fx /api/skelpunkter.
- Browserkoden kalder kun /api/skelpunkter — aldrig den rigtige, hemmelige URL direkte.
- Vite-serveren sender kaldet videre til den rigtige tjeneste med nøglen indsat.

Nogle API'er (som DAWA-adressesøgningen) er derimod lavet til at blive kaldt direkte fra browseren og kræver ingen nøgle eller proxy.

9. GraphQL — grundbegreber

GraphQL er en forespørgselssprog for API'er, hvor du selv beskriver præcis, hvilke felter du vil have — modsat traditionelle API'er, der altid sender alt. Et kald sendes som en POST-forespørgsel med en tekststreng ("query"), der beskriver ønsket.

```
query {  
  MAT_Skelpunkt(first: 1000, where: { status: { eq: "Gældende" } }) {  
    nodes { id_lokalId geometri { wkt } }  
  }  
}
```

Vigtige begreber, vi stødte på:

- **Schema** — en fil der beskriver alle tilgængelige felter og typer i API'et.
- **Filter (where)** — indsnævrer resultatet, fx til et bestemt geografisk område.
- **WKT** (Well-Known Text) — en tekstlig måde at beskrive geometrier på, fx "POINT (12.5 55.6)".
- **pageInfo / hasNextPage** — fortæller om der findes flere resultater, end du fik med.
- Nogle servere tillader kun ét "rod-felt" pr. forespørgsel — så flere datatyper skal hentes i separate kald.

10. Projektstruktur: utils vs. components

Efterhånden som appen voksede, delte vi den op for at holde den overskuelig:

```
src/  
  utils/ — ren logik, ingen skærm-visning (matematik, API-kald)  
    coordinates.js — koordinatkonverteringer og areal/omkreds  
    skelApi.js — GraphQL-kald til Dataforsyningen  
    addressSearch.js — DAWA-adressesøgning  
  components/ — visuelle byggeklodser  
    MapView.jsx, PointsTable.jsx, AddressSearch.jsx m.fl.  
  App.jsx — samler det hele, holder styr på state
```

Tommelfingerregel: Hvis koden regner noget ud eller henter data, hører den til i utils/. Hvis den viser noget på skærmen, hører den til i components/.

11. Fejlfinding — typiske faldgruber

Symptom	Sandsynlig årsag
npm error ENOENT ... package.json	Du står i den forkerte mappe — cd ind i surveyor-app
Ændringer i vite.config.js virker ikke	Husk at genstarte npm run dev efter ændringer i config/.env
401 Unauthorized fra et API	Forkert/manglende API-nøgle, eller forkert parameternavn
400 Bad Request fra GraphQL	Fejl i selve forespørgslens syntaks — læs fejlbeskeden i svaret
Intet vises, men ingen fejl i konsollen	Data kan være tomt/ugyldigt (fx NaN-koordinater) uden at kaste en fejl — tilføj console.log
CORS-fejl i browserens konsol	API'et tillader ikke direkte browserkald — brug en proxy (se afsnit 8)

God vane: Når noget ikke virker, og I ikke kan se hvorfor — tilføj midlertidige console.log-linjer for at se, hvad koden faktisk modtager og regner med. Fjern dem igen, når fejlen er fundet.

12. Kommando-opslag (hurtig reference)

Formål	Kommando
Start udviklingsserver	<code>npm run dev</code>
Installer en pakke	<code>npm install <pakkenavn></code>
Gem ændringer i Git	<code>git add . && git commit -m "besked"</code>
Send til GitHub	<code>git push</code>
Se Git-historik	<code>git log --oneline</code>
Se ændrede filer	<code>git status</code>

13. Ordliste

API — Et "kontaktpunkt", hvor en applikation kan hente eller sende data til en anden tjeneste.

Component / komponent — En genbrugelig byggeklods i React, der viser noget på skærmen.

CORS — En browser-sikkerhedsregel, der styrer hvilke websider et API må kaldes direkte fra.

Debounce — At vente lidt, før man reagerer på gentagne hændelser (fx tastetryk), for ikke at lave for mange kald.

Endpoint — Den specifikke webadresse, et API-kald sendes til.

Geocoding — At omsætte en adresse til koordinater (bruges i adressesøgningen).

Hook (React) — En speciel funktion (fx `useState`, `useEffect`) der giver komponenter ekstra evner.

JSX — HTML-lignende syntaks brugt inde i JavaScript til at beskrive, hvad React skal vise.

Props — Data sendt fra en forælder-komponent til en barn-komponent.

Proxy — Et mellemlid der videresender kald og kan skjule hemmeligheder undervejs.

Repository (repo) — En Git-sporet projektmappe.

State — Data en komponent husker, og som appen genindlæses ud fra, når det ændrer sig.

WKT — Well-Known Text — tekstformat til at beskrive geometriske former (punkter, linjer, flader).